Technical Assignment

Full Stack Developer (React, Python, MongoDB, RAG)

Candidate name		Date received	
Submission due	5 calendar days from the date received	Expected effort	12 to 16 hours

1. Before you start

Thank you for taking the time to do this. Most of the work at MGD TECH is client projects where one developer owns a feature end to end, from how the data is stored to what the user sees on screen. This assignment is a smaller version of that kind of work.

There is no trick in it and no single correct answer. We care more about the decisions you make, and how clearly you explain them, than about the number of features. If you run out of time, finish fewer things properly and tell us in the README what you left out. A smaller app that works well is better than a large one that only half works.

If something in this document is unclear, make a sensible assumption, write it down in your README and keep going. If you are actually blocked, reply to the email this assignment came with.

2. The problem

Some of the clients we work with are education consultancies. Their counsellors help students apply to universities in the UAE and then help them with the student visa. A large part of their day goes into answering the same questions again and again: *Which documents do I need for the entry permit? Is the medical test done before or after I land? Can I get a refund if my visa is rejected?* The answers are spread across long PDFs, policy notes and university handbooks.

We want a small internal tool for them. We are calling it **Counsellor Assist**. It should let them do three things:

1. An admin uploads reference documents (handbooks, checklists, fee policies).
2. A counsellor asks a question in plain English and gets an answer that is based only on those documents, along with where in the documents the answer came from.
3. Counsellors record student enquiries and track each one until it is closed.

3. Sample data

Please create at least **four** reference documents yourself, two to five pages each, as PDF or Markdown. For example: an admissions guide for a fictional university, a student visa checklist, a fee and refund policy and an accommodation FAQ. Fictional content is actually what we want. It lets us check that answers really come from your documents and not from the model's general knowledge, so add a few specific details (made-up fees, deadlines, office hours) that a model could not guess.

Put the documents in a `sample_data/` folder in the repository. Also include a seed script that creates at least 30 realistic enquiries spread over the last two months, so the list and the dashboard have something to show.

4. What to build

4.1 Backend (Python)

- FastAPI is preferred. Flask or Django are fine if you tell us why you picked them.
- Login with JWT and two roles: **admin** and **counsellor**. Store passwords hashed. The seed script should create one user of each role.
- Uploading a document should return straight away. Parsing, chunking and embedding happen in the background, and the document carries a status: `pending`, `processing`, `ready` or `failed` (with the reason).
- Request and response validation with Pydantic, sensible HTTP status codes and one consistent shape for error responses.
- All configuration through environment variables. Commit a `.env.example`. No API keys in the repository, including in the git history.
- Some tests with pytest. We are not looking for full coverage. Test the parts you think are most likely to break, and tell us why you chose them.

The endpoints below are the minimum. You can rename them or add more.

Method and path	Who	What it does
`POST /auth/login`	all	Returns a token and the user's role.
`POST /documents`	admin	Upload a PDF or Markdown file. Processing runs in the background.
`GET /documents`	all	List documents with status, page count and chunk count.
`DELETE /documents/{id}`	admin	Delete a document and all of its chunks and vectors.
`POST /chat/ask`	all	Body: question, optional conversation_id, optional list of document ids to search in. Returns the answer, the sources (document, page, short snippet) and the conversation_id.
`GET /conversations`	all	The current user's conversations, newest first.
`GET /conversations/{id}`	all	Full message history of one conversation, with sources.
`POST /enquiries`	all	Create an enquiry.
`GET /enquiries`	all	Filter by status, course, nationality, assigned counsellor, date range and free-text search on name or email. Paginated.
`PATCH /enquiries/{id}`	all	Update fields or status, or add a note.
`GET /stats/enquiries`	all	Numbers for the dashboard (see 4.2).

4.2 MongoDB

We would like to see how you model data, not only how you query it. You decide the collections. In the README, show one example document per collection and explain where you chose to embed and where you chose to reference, and why.

An enquiry needs at least:

- student name, email, phone and nationality
- the course they are interested in and the intake (for example September 2027)
- a status, one of: new, contacted, documents pending, applied, visa in process, closed

- the assigned counsellor
- a history of notes, each with its author and time
- a history of status changes, each with the time it happened
- created and updated timestamps

Other requirements:

- Create the indexes your queries need. List them in the README with one line on what each one is for.
- `GET /stats/enquiries` must be computed with aggregation pipelines in MongoDB, not by loading documents into Python and looping. It should return: the count per status, the number of new enquiries per week for the last 8 weeks, the top 5 courses, and the average number of days from *new* to *applied* for enquiries that reached *applied*.
- Choose skip and limit or cursor-based pagination and say why in a sentence or two.
- For vectors, MongoDB Atlas Vector Search (the free tier is enough) is preferred. If you use something else, such as Chroma, FAISS or Qdrant, keep the document and chunk metadata in MongoDB anyway and explain the trade-off.

4.3 RAG pipeline

- Extract text from PDFs and keep page numbers, because the answer has to point to them.
- Pick a chunking approach. In the README, tell us the chunk size and overlap you ended up with and what made you choose them.
- Use any embedding model (a hosted API or a local one such as sentence-transformers) and any LLM (OpenAI, Anthropic, Gemini, Groq or a local model through Ollama). Changing the provider should only need a config change.
- Answers must stay inside the documents. When the documents do not contain the answer, the assistant should say so instead of guessing. We will test this on purpose.
- Every answer comes back with its sources. The source shown should be the passage the answer actually came from, not just the top search result.
- Follow-up questions in the same conversation should work. For example, after asking about visa documents, the counsellor asks *"and what about diploma students?"*
- LangChain or LlamaIndex are fine for small pieces such as file loaders or text splitters. Please do not hide the whole pipeline behind one high-level call. We should be able to read chunking, retrieval and prompt building in your own code.

A small evaluation. Add an `eval/` folder with:

- a `questions.json` file with at least 12 questions you wrote yourself. For each one, record the document and page where the answer is. At least 3 of the questions should not be answerable from your documents.
- a script that runs all of them and prints two numbers: how often the correct page appears in the top *k* retrieved chunks, and how many of the unanswerable questions were correctly refused.

Please report the numbers honestly, even if they are low. An honest 60 percent with a clear note about why is more useful to us than a 100 percent we cannot reproduce.

4.4 Frontend (React)

- React 18 or later with Vite. TypeScript is preferred. Use any component library, or plain CSS.
- **Login** screen. Once logged in, counsellors should not see admin actions at all, and the backend must reject them too.
- **Documents** (admin): upload with progress, a list whose status updates without a manual page refresh, and delete with a confirmation.
- **Ask**: a chat screen with a list of past conversations. Each answer shows its sources, and clicking a source shows the passage it came from. Handle the loading state, errors and the case where the documents have no answer.
- **Enquiries**: a table with filters and pagination. Filters should be kept in the URL, so a filtered view can be bookmarked or shared. A create and edit form with validation, a way to add notes and change status, and a view of the history.
- **Dashboard**: the numbers from the stats endpoint, with at least one chart.
- It should work well on a laptop and still be usable at tablet width. We do not expect a designer's finish, but it should look tidy and consistent.

5. If you have time left

None of these are required. Please only start on them once the core is done.

- `docker compose up` starts everything.
- Answers stream into the chat word by word (Server-Sent Events or WebSockets).
- Hybrid search (keyword plus vector) or a re-ranking step, with before and after numbers from your evaluation script.
- A CI pipeline that runs the tests on every push.
- A deployed version we can open in the browser.

6. What to send us

1. A link to the Git repository. A private repository is fine, just give us access. Please commit as you go. We do look at the history, so do not squash it into a single commit.
2. A README that covers: setup steps that work on a clean machine, a simple architecture diagram (a photo of a hand drawing is perfectly fine), the data model, your RAG choices, the evaluation results, known problems and what you would do next with another week.
3. A screen recording of 5 to 8 minutes. Walk us through the app, then open one part of the code you are happy with and explain it. Loom or a Google Drive link is fine.
4. Roughly how many hours you spent.

7. Using AI tools

We use tools like Copilot, ChatGPT and Claude ourselves, so you are welcome to use them. Mention in the README which ones you used and for what. After you submit, we will have a call where we go through your code together and ask you to make a small change live. You should be able to explain any line in the repository, whoever or whatever typed it.

8. How we will review it

This is roughly how we will weigh things:

Area	Weight	What we are looking at
Backend and API	25%	Structure, validation, auth and roles, error handling, background processing, tests.
MongoDB	20%	Schema decisions, indexes, aggregation pipelines, pagination.
RAG	25%	Chunking, retrieval quality, grounded answers, correct sources, refusing when the answer is not there, evaluation.
Frontend	20%	Component structure, state and data fetching, the user experience on each screen, role handling.
Code quality and communication	10%	Readability, commit history, README, the video and how well you explain your trade-offs.

After the review there is a call of about an hour. You will demo the app, take us through your decisions and make one or two small changes with us. Good luck, and we look forward to seeing what you build.